

L17: Non-Homogeneous Poisson Arrivals

Simulation Without a Black Box

Outline

Learning Objectives

Time-Varying Arrivals

NHPP Definition

Thinning Algorithm

Piecewise-Constant Approximation

SimPy Thinning Implementation

Call Centre Example

NHPP vs. Multiple Steady States

Key Takeaways

Learning Objectives

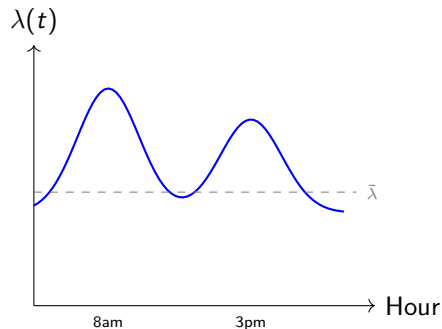
By the end of this lecture you will be able to:

- Explain why real arrival processes are rarely stationary.
- Define the Non-Homogeneous Poisson Process (NHPP) via its intensity function $\lambda(t)$.
- Implement the Lewis-Shedler thinning algorithm in SimPy.
- Approximate $\lambda(t)$ with a piecewise-constant step function.
- Decide when to use a full NHPP vs. multiple steady-state simulation periods.

Real Data Shows Time-Varying Arrival Rates

- **Call centres:** morning rush 8–10 am; lunch dip; afternoon peak 2–4 pm.
- **Emergency departments:** peaks in early morning and evening.
- **Retail:** weekend spikes; seasonal patterns.
- **Network traffic:** hourly, daily, and weekly cycles.

Using a stationary Poisson model with the average $\bar{\lambda}$ systematically understates peak congestion and overstates off-peak congestion.



Non-Homogeneous Poisson Process: Definition

NHPP

A counting process $N(t)$ is an NHPP with intensity function $\lambda(t) \geq 0$ if:

1. $N(0) = 0$
2. Increments are independent: $N(s, t]$ independent of $N(0, s]$ for $s < t$
3. $P[N(t, t + h) = 1] = \lambda(t)h + o(h)$ as $h \rightarrow 0$

Mean number of arrivals in $[t_1, t_2]$:

$$E[N(t_1, t_2)] = \int_{t_1}^{t_2} \lambda(u) du = \Lambda(t_1, t_2)$$

For a stationary PP: $\lambda(t) \equiv \lambda \Rightarrow \text{NHPP} \supset \text{HPP}$.

- Inter-arrival times of an NHPP are *not* i.i.d. — they depend on t .
- Direct inversion of inter-arrival CDF requires solving $\Lambda(t, t + X) = E$ — usually numerical.

Lewis-Shedler Thinning Algorithm

Thinning (acceptance-rejection for NHPP)

1. Find $\lambda^* = \max_{t \in [0, T]} \lambda(t)$ (majorising rate).
2. Generate candidate arrivals from a stationary PP at rate λ^* .
3. Accept each candidate at time t with probability $\lambda(t)/\lambda^*$; reject otherwise.
4. Accepted events form an NHPP with intensity $\lambda(t)$.

Why it works: Poisson thinning theorem — independently thinning a PP with probability p yields a PP with rate λp .

- Simple to implement; no numerical inversion needed.
- Efficiency: $\bar{\lambda}/\lambda^*$ fraction of candidates accepted; good when $\lambda(t)$ is smooth.

Piecewise-Constant $\lambda(t)$ as Approximation

```
import numpy as np

# Hourly arrival rates (24-hour day, hours 0..23)
HOURLY_RATES = [
    5, 3, 2, 2, 3, 8, 20, 35, 42, 38, 30, 28,
    32, 30, 36, 42, 38, 28, 20, 15, 12, 10, 8, 6
]

def piecewise_lambda(t):
    """Return arrival rate at time t (minutes from midnight)."""
    hour = int(t / 60) % 24
    return HOURLY_RATES[hour] / 60    # convert hr-1 to min-1
```

- Use 15-min or 30-min buckets when hourly is too coarse.
- Fit rates from historical data: $\hat{\lambda}_k = n_k / (\Delta t \cdot n_{\text{days}})$.
- The piecewise approximation is itself an NHPP — no continuity required.

SimPy Thinning Implementation

```
import simpy, numpy as np

LAMBDA_MAX = max(HOURLY_RATES) / 60    # majorising rate

def nhpp_source(env, server, rng, stats):
    while True:
        # Step 1: generate candidate from HPP at lambda_max
        yield env.timeout(rng.exponential(1/LAMBDA_MAX))

        # Step 2: thin with acceptance probability
        lam_t = piecewise_lambda(env.now)
        if rng.random() < lam_t / LAMBDA_MAX:
            # accepted: spawn a customer process
            env.process(customer(env, server, rng, stats))
        # rejected: loop back for next candidate
```

- Thinning adds at most $O(\lambda^*/\bar{\lambda})$ extra event evaluations per unit time.
- For a 10:1 peak-to-trough ratio, 90% of candidates are rejected — still correct.

Example: Call Centre with Morning and Afternoon Peaks

Setup: 8-hour shift, $\lambda(t)$ rises to 60 calls/hr at 9 am, dips to 20 at noon, peaks at 50 at 3 pm.

Period	λ (calls/hr)	ρ (3 agents)
8–9 am	30	0.50
9–11 am	60	1.00
12–1 pm	20	0.33
1–3 pm	45	0.75
3–5 pm	50	0.83

Steady-state fallacy

Using the average $\bar{\lambda} = 45$ hides the 9–11 am overload ($\rho = 1.0$). NHPP simulation correctly captures transient congestion during the peak.

Warm-up: discard first 30 minutes of simulated time (system starts empty, not at steady state).

When to Use NHPP vs. Multiple Steady-State Periods

Use NHPP when

- Rate changes are *gradual* (smooth $\lambda(t)$).
- System takes a long time to reach steady state relative to the rate-change period.
- Transient behaviour (e.g., morning build-up) is itself of interest.
- You have sufficient data to estimate $\lambda(t)$ at fine resolution.

Use multiple steady-state periods when

- Rate changes are *abrupt* (step changes between shifts).
- Each period is long relative to the queue's equilibration time.
- Simpler to implement and explain.
- Analytical results exist for each static period.

Rule of thumb: if $1/(\mu - \lambda) \ll \text{period length}$, steady-state analysis per period is reliable.

Key Takeaways

Core ideas from L17

1. Real arrival processes are rarely stationary — using average $\bar{\lambda}$ masks peak congestion.
2. An **NHPP** has intensity $\lambda(t)$; mean arrivals in $[t_1, t_2]$ is $\int_{t_1}^{t_2} \lambda(u) du$.
3. **Lewis-Shedler thinning**: generate at λ^* , accept at probability $\lambda(t)/\lambda^*$ — no inversion needed.
4. Piecewise-constant $\lambda(t)$ is easy to fit from historical data and directly usable in thinning.
5. Use NHPP for gradual variation; use multiple steady-state periods for abrupt shift changes.

Next: L18 — Probabilistic and attribute-based routing in service networks.